Project Learning & Fresher Interview Preparation Report

Project Name: CRAVINGZA

Application Type: Full-Stack Multi-Role Food Ordering & Delivery Platform

Target Audience: Fresher Software Engineer / Full-Stack Developer Candidates

---

1. PROJECT OVERVIEW

Simple Explanation

CRAVINGZA is a complete online food ordering and delivery system (like Swiggy or Zomato). It connects four main types of users:

1. **Customers** who browse food menus, add items to cart, order online via Razorpay or COD, and track their delivery.

2. **Restaurant Owners** who manage their restaurant profile, menu items, prices, and accept/prepare customer orders.

3. **Delivery Partners (Riders)** who view nearby ready orders, accept deliveries, update live status, and earn payouts.

4. **Admins** who review & approve restaurant/rider applications, manage user accounts, monitor commission earnings, and control platform settings.

Technical Explanation

CRAVINGZA is architected as a modern client-server web application. It features a decoupled **RESTful API Backend** built with **Node.js, Express.js, and MongoDB (Mongoose)** and a **Server-Side Rendered (SSR) / React Client** built with **Next.js 16 (App Router), React 19, TypeScript, and Redux Toolkit (RTK Query)**.

User Roles & Permissions:

- `customer`: Browse restaurants, manage address book, cart state, place orders, write reviews, edit profile.

- `restaurant_owner`: Manage menu CRUD, restaurant status/settings, view incoming orders, update cooking stages.

- `delivery_partner`: Submit onboarding docs, view available ready orders, accept order, update status (`picked_up`, `delivered`), view earnings.

- `admin`: Platform governance, partner onboarding approval/rejection, commission configuration, system-wide metrics.

End-to-End System Flows:

1. **Frontend Flow:** Next.js App Router renders pages organized by role-based route groups `(customer)`, `(restaurant-owner)`, `(delivery-partner)`, `(admin)`. State is managed via Redux Toolkit (`apiSlice.ts`) for server cache and Zustand/Local state for UI modals.

2. **Backend Flow:** Express application routing HTTP requests through CORS, Helmet security headers, Rate Limiters, Cookie Parsers, and JWT Authentication Middleware (`auth.js`) to Controller functions. Controllers validate input via Zod schemas, execute database queries via Mongoose Models, handle transactions/side-effects, and return structured JSON responses.

3. **Frontend-Backend Communication:** Asynchronous HTTPS requests using RTK Query / Fetch API with `credentials: "include"` for HTTP-only JWT cookies.

4. **Database Flow:** MongoDB document store accessed asynchronously using Mongoose ODM. Schemas enforce type validation, pre-save hooks (e.g., Bcrypt password hashing in `User.js`), virtual populates, and indexing.

---

2. TECHNOLOGIES USED

The following technologies and libraries are **ACTUALLY present and used** in the CRAVINGZA codebase:

Technolog	Where Used	Why	Example from My Codebase

y / Library		Used	
Next.js 16	`my-next-app/src/app`	Server-Side Rendering (SSR), App Router, file-based routing, SEO optimization.	`my-next-app/src/app/(customer)/profile/page.tsx`
React 19	`my-next-app`	Building interactive UI component hierarchy and managing state.	`useState`, `useEffect` across components
TypeScript	`my-next-app/src`	Type safety, interface definitions, catching compile-time errors.	`interface NotificationPreferences` in `profile/page.tsx`
Tailwind CSS v4	`my-next-app/src/app/globals.css`	Utility-first responsive styling and dynamic design system tokens.	`className="bg-white rounded-2xl shadow-2xl"`
Node.js	`backend/`	JavaScript runtime environm	`backend/index.js` server entry point

		ent executing backend server logic.	
Express.js	`backend/index.js`, `routes/`	Web framework for building REST APIs, routing, and middleware pipelines.	`const app = express(); app.use('/api/user', userRoutes)`
MongoDB & Mongoose	`backend/config/db.js`, `models/`	NoSQL document database and Object Data Modeling (ODM) library.	`mongoose.connect(process.env.MONGO_URI)` in `db.js`
REST API	`backend/routes/`, `apiSlice.ts`	Standardized HTTP protocol communication (`GET`, `POST`, `PATCH`, `DELETE`).	`router.patch('/profile', protect, updateProfile)`
Redux Toolkit (RTK Query)	`my-next-app/src/lib/redux/apiSlice.ts`	Global API state management, auto-caching, polling,	`useUpdateProfileMutation()`, `invalidatesTags: ['User']`

		and cache invalidation.	
Zustand	`my-next-app/package.json`	Lightweight client-side state management for UI states.	Imported in frontend utilities
JWT (JsonWebToken)	`backend/controllers/authController.js`	Stateless secure authentication token encoding user ID & role.	`jwt.sign({ userId: user._id }, process.env.JWT_SECRET)`
Cookies (`cookie-parser`)	`backend/middlewares/auth.js`	Storing JWT securely in HTTP-Only cookies to protect against XSS attacks.	`res.cookie('token', token, { httpOnly: true, secure: true })`
Bcryptjs	`backend/models/User.js`, `authController.js`	One-way salt hashing for passwords.	`userSchema.pre('save', async function() { this.password = await bcrypt.hash(...) })`
Zod	`backend/validators/`, `userController.js`	Strict runtime schema validation for incoming	`profileSchema.safeParse(req.body)`

		client request payloads.	
**Multer & Streamifier **	`backend/ routes/ uploadRoutes.j s`	Handling multipart/f orm-data image uploads in memory buffers.	`multer({ storage: multer.memoryStorage() })`
Cloudinar y	`backend/ utils/ cloudinary.js`	Cloud media storage service for image hosting and dynamic transform ation.	`cloudinary.uploader.upload_stream(...)`
Razorpay SDK	`backend/ controllers/ paymentContro ller.js`	Processin g online payments , verifying HMAC SHA256 payment signature s.	`razorpay.orders.create(...)`, `crypto.createHmac(...)`
Brevo (@getbrev o/brevo)	`backend/ services/ emailService.js `	Transacti onal email delivery service for sending verificatio n OTPs.	`TransactionalEmailsApi.sendTransacEm ail(...)`
Leaflet & React-Leaflet	`my-next-app/ package.json`	Interactiv e map rendering	Leaflet map markers for customer delivery addresses

		for delivery location coordinates.	
Firebase / Web-Push	`backend/config/firebase.js`, `web-push`	Sending push notifications for live order updates.	`webpush.sendNotification(subscription, payload)`
Helmet	`backend/index.js`	Securing Express app by setting security-related HTTP headers.	`app.use(helmet())`
CORS	`backend/index.js`	Configuring Cross-Origin Resource Sharing for trusted frontend origin.	`app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }))`

---

3. FOLDER STRUCTURE

Backend Folder Structure (`/backend`)

```

backend/

├── config/ # Infrastructure configurations

│ ├── db.js # MongoDB connection using Mongoose
```

```
│  ├── firebase.js # Firebase Admin SDK initialization
│  └── swagger.js # Swagger API documentation configuration
├── controllers/ # Request handling and business logic
│  ├── adminController.js # Platform administration, approvals, payouts
│  ├── authController.js # Registration, OTP verification, Login, Logout
│  ├── cartController.js # Cart add/remove/sync logic
│  ├── deliveryController.js# Rider dashboard, order pickup/delivery status
│  ├── menuController.js # Restaurant menu item CRUD operations
│  ├── orderController.js # Order placement, status pipeline, refunds
│  ├── paymentController.js # Razorpay order creation and signature verification
│  └── userController.js # Profile management, address book, preferences
├── middlewares/ # Express Request Interceptors
│  ├── auth.js # Token extraction from cookie/header, role authorization
│  └── rateLimiter.js # IP rate limiting for API protection
├── models/ # Mongoose Schemas & Database Entities
│  ├── User.js # User account entity (customer/owner/rider/admin)
│  ├── Restaurant.js # Restaurant store details & status
│  ├── MenuItem.js # Dish name, price, category, veg/non-veg flag
│  ├── Cart.js # Customer persistent shopping cart
│  ├── Order.js # Order details, timeline, financial breakdown
│  ├── DeliveryProfile.js # Delivery partner documents & vehicle details
│  ├── Delivery.js # Delivery tracking & assignment entity
│  ├── Coupon.js # Promotional discount offers
```

```
│ ├── Review.js # Restaurant & dish rating entity
│ ├── Notification.js # In-app user notifications
│ └── SystemSettings.js # Global commission rates & operational settings
├── routes/ # Express Router declarations mapped to endpoints
│ ├── authRoutes.js # /api/auth endpoints
│ ├── userRoutes.js # /api/user endpoints
│ ├── orderRoutes.js # /api/orders endpoints
│ └── uploadRoutes.js # /api/upload Cloudinary image upload endpoint
├── services/ # External Service Integrations
│ └── emailService.js # Brevo API email sending logic
├── utils/ # Helper Utilities
│ ├── cloudinary.js # Cloudinary upload stream helper
│ └── otp.js # Crypto-secure random 6-digit OTP generator
├── validators/ # Input Schema Validations
│ ├── authValidator.js # Zod schemas for signup/login
│ └── shared.js # Shared pincode & phone validators
└── index.js # Express Application Bootstrap & Server Listener
```

Frontend Folder Structure (`/my-next-app`)

```
my-next-app/
├── src/
│ ├── app/ # Next.js 16 App Router Directory
```

```
│ │ ├── (admin)/ # Route Group: Admin Dashboard & Approval Views
│ │ ├── (customer)/ # Route Group: Home, Restaurants, Cart, Checkout, Profile
│ │ ├── (delivery-partner)/ # Route Group: Rider Dashboard & Active Delivery Views
│ │ ├── (restaurant-owner)/ # Route Group: Owner Dashboard, Menu, Settings
│ │ ├── (public)/ # Route Group: Landing Page, Login, Register
│ │ ├── globals.css # Tailwind CSS base styles & custom properties
│ │ └── layout.tsx # Root HTML Layout wrapper
│ ├── components/ # Reusable UI Components
│ │ ├── AuthInitializer.tsx # Silent session restoration component
│ │ ├── ProfileMenu.tsx # User account dropdown menu
│ │ ├── ReduxProvider.tsx # Client wrapper for Redux store
│ │ └── customer/ # Customer-specific components (Cards, Modals)
│ ├── lib/ # Core utilities & state configuration
│ │ ├── redux/
│ │ │ └── apiSlice.ts # RTK Query centralized API service layer
│ │ ├── store.ts # Redux Store configuration
│ │ └── firebase.ts # Client Firebase Messaging setup
└── package.json # Frontend dependency manifest
```

---

4. CONCEPTS LEARNED FROM THIS PROJECT

A. React Concepts

1. **Component-Driven Architecture**

- *Definition:* Building UI using modular, reusable independent pieces.

- *Where Used:* `EditProfileModal` component in `profile/page.tsx`.

- *Interview Answer:* "I built modular React components that isolate presentation and business logic, promoting maintainability."

2. **Hooks (`useState`, `useEffect`, `useRef`)**

- *Definition:* React functions allowing state and lifecycle management in functional components.

- *Where Used:* Managing modal visibility and input state in `profile/page.tsx`.

- *Code Example:*

```tsx
const [modals, setModals] = useState({ profile: false, password: false });
```

B. Next.js Concepts

1. **App Router & Route Groups**

- *Definition:* Next.js folder-based routing using parentheses `(group)` to organize routes without affecting the URL path.

- *Where Used:* Separation of `(customer)`, `(admin)`, and `(restaurant-owner)` layouts.

- *Interview Answer:* "Route groups allowed me to apply custom layouts to admin and customer sections independently without polluting URL routes."

C. Backend & Express Concepts

1. **Middleware Pipeline**

- *Definition:* Functions that intercept incoming requests before reaching controller handlers to perform authentication, parsing, or logging.

- *Where Used:* JWT extraction and role checking in `auth.js`.

- *Code Example:*

```javascript

const protect = async (req, res, next) => {

const token = req.cookies.token;

if (!token) return res.status(401).json({ message: "Not authorized" });

const decoded = jwt.verify(token, process.env.JWT_SECRET);

req.user = await User.findById(decoded.userId);

next();

};

```

D. Authentication & Security Concepts

1. **HTTP-Only Cookie Storage**

- *Definition:* Storing sensitive tokens in cookies configured with `httpOnly: true` so client-side JavaScript cannot read them.

- *Why Used:* Prevents Cross-Site Scripting (XSS) attackers from stealing session tokens.

- *Where Used:* `authController.js`.

2. **Input Validation with Zod**

- *Definition:* Validating data schemas at runtime before processing backend business logic.

- *Where Used:* Profile & password schema validation in `userController.js`.

---

5. IMPORTANT CODE FLOWS

Flow 1: User Profile Update (`PATCH /api/user/profile`)

```

User clicks "Edit Profile" on UI

↓

EditProfileModal opens with pre-filled inputs

↓

User submits Form (Name & Phone)

↓

handleProfileSave() triggers RTK Query useUpdateProfileMutation()

↓

HTTP PATCH Request with Credentials (HTTP-Only Cookie) sent to /api/user/profile

↓

Backend Express App receives request → Passes through auth.js Middleware (Verifies JWT)

↓

Request reaches updateProfile Controller in userController.js

↓

Zod schema profileSchema.safeParse(req.body) validates name length & phone format

↓

Mongoose model User.findById(req.user._id) fetches document

↓
```

Document fields updated & user.save() persists changes to MongoDB

↓

HTTP 200 OK Response returned with updated user data

↓

RTK Query cache invalidates "User" tag → UI automatically updates & displays Toast Success!

```

---

## 6. API DOCUMENTATION FOR MY OWN UNDERSTANDING

| Method | Endpoint | Purpose | Authentication | Payload / Params | Response | Co |
|---|---|---|---|---|---|---|
| `POST` | `/api/auth/register` | Register new user & send OTP | Public | `{ name, email, password }` | `{ success: true, message }` | `au |
| `POST` | `/api/auth/verify-otp` | Verify 6-digit OTP & activate account | Public | `{ email, otp }` | `{ success: true, user }` | `au |
| `POST` | `/api/auth/login` | Login user & set HTTP-Only | Public | `{ email, password }` | `{ success: true, data }` | `au |
```

		JWT Cookie			`	
`POST`	`/api/auth/logout`	Clear JWT Cookie & destroy session	Authenticated	None	`{ success: true }`	`au
`PATCH`	`/api/user/profile`	Update profile Name and Phone	Authenticated	`{ name, phone }`	`{ success: true, data }`	`us
`POST`	`/api/orders`	Place order (COD or Razorpay)	Customer	`{ items, deliveryAddress, paymentMethod }`	`{ success: true, data }`	`or
`POST`	`/api/payments/create-razorpay-order`	Generate Razorpay order ID	Customer	`{ orderId }`	`{ success: true, razorpayOrderId }`	`pa Ord
`POST`	`/api/upload`	Upload image buffer to Cloudi	Authenticated	`FormData (file)`	`{ success: true, url }`	`up

		nary				

---

7. AUTHENTICATION & SECURITY

Implemented Features:

1. **Password Hashing:** Passwords hashed with `bcryptjs` using salt round 12 via Mongoose pre-save hook in `User.js`.

2. **JWT in HTTP-Only Cookies:** Tokens stored in `httpOnly` cookies with `sameSite: 'lax'` or `'none'` and `secure: true` in production.

3. **Role-Based Access Control (RBAC):** Middleware checks `req.user.role` against required roles (`admin`, `restaurant_owner`, `delivery_partner`).

4. **Input Sanitization & Validation:** Zod schemas prevent injection of invalid data payloads.

5. **Security Headers:** Express app secured using `helmet()` middleware.

---

8. DATABASE MODELS & MONGOOSE SCHEMAS

CRAVINGZA includes **11 Mongoose Data Models**:

1. **User Schema (`User.js`):** Stores credentials, role (`customer`, `restaurant_owner`, `delivery_partner`, `admin`), addresses array subdocument, OTP verification state, and notification preferences.

2. **Restaurant Schema (`Restaurant.js`):** Stores restaurant info, owner reference (`ref: 'User'`), address, geolocation, opening status, cuisine tags, and admin approval status (`pending`, `approved`, `rejected`).

3. **Order Schema (`Order.js`):** References `customer`, `restaurant`, `deliveryPartner`. Contains items array, delivery address, financial breakdown (subtotal, delivery fee, taxes, admin commission), order status timeline (`placed` → `accepted` → `preparing` → `ready_for_pickup` → `picked_up` → `delivered`), and Razorpay payment details.

4. **MenuItem Schema (`MenuItem.js`):** Dish details linked to a `restaurant`. Includes title, description, price, image URL, category, availability flag, and vegetarian indicator.

5. **Cart Schema (`Cart.js`):** Persistent customer cart storing items array linked to a specific restaurant.

6. **DeliveryProfile Schema (`DeliveryProfile.js`):** Stores delivery partner documents (DL, Vehicle Registration), vehicle type, approval status, and earnings.

---

9. FRONTEND & BACKEND ARCHITECTURE SUMMARY

* **Frontend:** Built with Next.js 16 App Router using Client Components (`"use client"`) for interactivity, Redux Toolkit for cached data fetching, and Tailwind CSS for custom styling.

* **Backend:** Express.js classic **MVC Architecture** (Model-View-Controller). Requests hit routes -> pass auth middleware -> trigger controller functions -> interact with Mongoose models -> query MongoDB -> format standardized JSON response.

---

10. GIT & GITHUB WORKFLOW

Standard industry Git workflow followed:

```

main (Production deployment branch)

└── staging (Integration testing branch)

└── feature/user-profile-update (Feature development branch)

```

Commands: `git checkout -b feature/name` → `git add .` → `git commit -m "feat: description"` → `git push origin feature/name` → Open Pull Request (PR) for review → Merge to `staging`.

---

11. FRESHER INTERVIEW QUESTIONS & ANSWERS (TOP 10 SAMPLES)

1. **Q: How does authentication work in your CRAVINGZA project?**

* *Short Answer:* "I implemented JWT authentication stored in HTTP-Only cookies. When a user logs in, the backend generates a JWT token and sends it in a secure HTTP-Only cookie. Subsequent requests automatically send this cookie, which my Express `auth` middleware verifies."

2. **Q: Why did you use HTTP-Only cookies instead of LocalStorage for storing JWTs?**

* *Short Answer:* "LocalStorage is accessible by client-side JavaScript, making it vulnerable to Cross-Site Scripting (XSS) attacks. HTTP-Only cookies cannot be accessed by scripts, securing the session token against theft."

3. **Q: How do you handle input validation on the backend?**

* *Short Answer:* "I used Zod schemas to validate incoming request data in the controllers before executing any business logic or database queries. If validation fails, a 400 Bad Request with formatted error details is returned."

4. **Q: What state management solution did you use on the frontend?**

* *Short Answer:* "I used Redux Toolkit with RTK Query (`apiSlice.ts`) for centralized server state management, automated caching, and cache invalidation. For local UI state like modal visibility, I used React's `useState` hook."

5. **Q: How do you handle image uploads in your application?**

* *Short Answer:* "I configured Multer memory storage on Express to buffer image files, and then streamed the image buffer directly to Cloudinary using their SDK upload stream helper. The returned Cloudinary image URL is then saved in MongoDB."

---

12. FINAL CHEAT SHEET

* **Project:** CRAVINGZA (Multi-Role Online Food Delivery Platform)

* **Stack:** Node.js, Express.js, MongoDB, Mongoose, Next.js 16, React 19, TypeScript, Tailwind CSS, RTK Query, Zod, Razorpay, Cloudinary.

* **Core API Endpoint for Profile:** `PATCH /api/user/profile` (Controller: `userController.js`, Handler: `updateProfile`).

* **Main Strengths:** Production-ready RBAC authentication, HTTP-only cookie security, Zod input validation, RTK Query caching, modular layout architecture.